



Problem Solving With C - UE25CS151B

Functions

Prof. Sindhu R Pai

Department of Computer Science and
Engineering

PROBLEM SOLVING WITH C

Functions



1. Introduction to functions
2. Types of Functions
3. Function Definition, Call and Declaration
4. Usage of return keyword
5. Parameter passing in C

Introduction

- A sub-program to carry out a specific task
- Functions break large computing tasks into smaller ones.
- Enable people to build on what others have done instead of starting from scratch
- **Benefits:**
 - Reduced Coding Time – Code Reusability
 - Divide and Conquer - Manageable Program development
 - Reduced Debugging time
 - Treated as a black box - The inner details of operation are invisible to rest of the program

Types of Functions

- **Standard Library Functions**

Must Include appropriate header files to use these functions.
Already declared and defined in C libraries
printf, scanf, etc..

- **User Defined functions**

Defined by the developer at the time of writing program
Developer can make changes in the implementation

Function Definition, Call and Declaration

Function Definition

- Provides actual body of the function
- Function definition format

```
return-type  function-name( parameter-list )
{
    declarations and statements
}
```
- Variables can be declared inside blocks
- Coding examples

Function Definition, Call and Declaration

Function Call

- Function can be called by using function name followed by list of arguments (if any) enclosed in parentheses
- Function call format
function-name(list of arguments);
- The arguments must match the parameters in the function definition in its type, order and number.
- Multiple arguments must be separated by comma. Arguments can be any expression in C
- Coding examples

Function Definition, Call and Declaration

Function Declaration/ Prototype

- All functions must be declared before they are invoked or called.
- Function can be declared by using function name followed by list of parameters (if any) enclosed in parentheses
- Function declaration format
return_type function_name (parameters list);
- Use of identifiers in the declaration is optional.
- The parameter names do not need to be the same in declaration and the function definition.
- The types must match the type of parameters in the function definition in number and order.
- Coding examples

Usage of return keyword

- return statement provides the means of exiting from a function and resuming execution of the calling function at the point from which the call occurred
- General form of return statement is
return expression;
- In 'C', function returns a single value. The expression of return is evaluated and copied to the temporary location by the called function. – No name
- If the return type of the function is not same as the expression of return in the function definition, the expression is cast to the return type before copying to the temporary location
- The value that's returned to the calling program is the value that results when expression is evaluated, and this should be of the return type specified for the function.
- Coding examples

Parameter passing in C

- **Parameter passing is always by value in C**
- Argument is copied to the corresponding parameter.
- Argument is not affected if we change the parameter inside a function.
- The formal parameter is not copied back to the actual argument. Possible only if the actual argument is l - value.
- Coding examples



THANK YOU

Department of Computer Science and Engineering

Dr. Shylaja S S, Director, CCBD & CDSAML, PESU

Prof. Sindhu R Pai - sindhurpai@pes.edu

Ack: Teaching Assistant - U Shivakumar